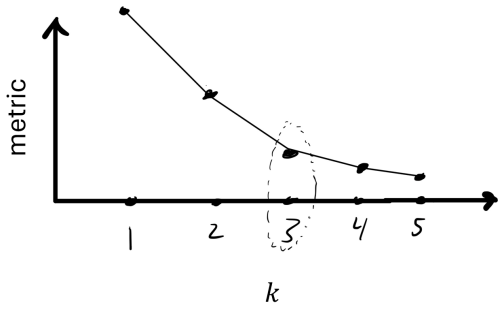
CS760 Final Exam

1. Please write your name and WISC ID/Email below.
2. **Do not turn this page until instructed to do so.**
3. You have 2 hours to answer all 8 questions.
4. Answer all the questions. You may write solutions below each question. If you do not have enough space, you can use the extra space on page 2 and the last page.
5. Read all the questions first. Do not spend too much time on any single problem. Some problems are harder than others.
6. Even if you cannot obtain a final answer, make sure to state your assumptions and set up, and explain how you would obtain the answer. Partial credit may be considered.
7. **Write only as much as needed to answer the question; the short answer questions that do not require derivations can typically be answered in 1-2 short sentences.**

Wisc ID: .....

Name: .....

If you do not write your information correctly your exam grade may be delayed.



1 Picking k

1.1 Learning algorithms [2.5 points]

Many ML algorithms have an integer hyperparameter $k > 0$ that significantly affects its performance. As discussed in class, often a good heuristic for tuning k is to look for a "knee" or "elbow" in the curve of some metric as a function of k. This is a point on the curve before which the curve is *steep* (decreasing rapidly) and after which it is *flat* (decreasing slowly). An example is circled in the picture above.

Below is a list of algorithms that take an integer hyperparameter. Circle all those for which picking k using a knee or elbow point is appropriate.

1. k-nearest-neighbors
2. decision trees of depth k
3. k-means clustering circled
4. k-component PCA circled
5. polynomial kernel of order k

1.2 Hyperparameter tuning strategies [5 points]

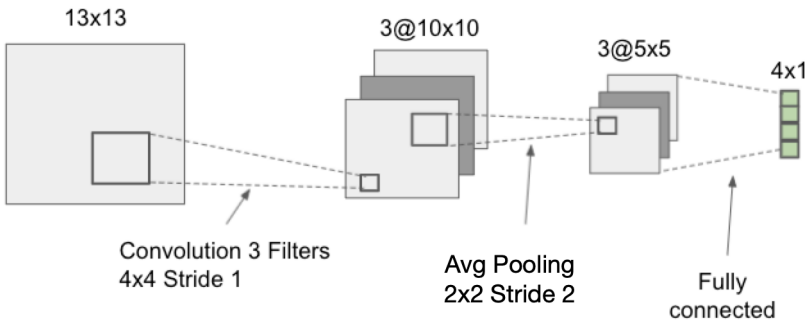
State one reason why it is NOT appropriate to look for a knee or elbow in the algorithms you did not circle in Question 1.1, AND one alternative way of picking k for those algorithms. Write ONE unified answer, NOT a separate answer for each method.

The uncircled algorithms are supervised learning algorithms, for which the metric of interest (test error) typically does not monotonically decrease with k. For such algorithms we usually use a held-out validation set or cross-validation to tune hyperparameters.

1.3 Setting the number of folds [5 points]

Unrelated to the previous two questions, suppose my learning pipeline involves (a) tuning a learning algorithm using k-fold cross-validation on the training dataset and then (b) retraining the model with the resulting configuration on the entire training dataset. State one advantage AND one disadvantage of increasing the value of k in k-fold cross-validation.

Increasing k increases the amount of training data given to the model trained on each individual fold and thus improves the aggregate estimate of the retraining performance. However, it also increases the computational cost of cross-validation, as each fold requires a single training run and k increases both the number of folds and the amount of training data per fold.



2 Convolutional neural networks

2.1 Filter weights [2.5 points]

Above is a diagram of a small CNN that converts a 13×13 image into 4 output values. The network has the following layers/operations from input to output: it first performs a 4×4 convolution with 3 filters (no activation after the convolution operation), then it performs average pooling, then it performs a ReLU operation, and finally it has a fully-connected layer. How many weights does the convolutional layer have?
48

2.2 Activation functions [2.5 poins]

How many (scalar) ReLU operations are performed in the forward pass of the CNN in Question 2.1?
75

2.3 Multi-channel convolutions [5 points]

Suppose I have an $h \times w$ input image with c_{in} channels. I pass it to a standard convolutional layer with kernel size $k \times k$ and obtain an $h \times w$ output with c_{out} channels. Using asymptotic notation, explain how much memory (number of parameters) is required to store the convolutional layer AND how compute is required to apply it to the input.

The convolutional layer has $c_{\text{in}}c_{\text{out}}$ filters of size $k \times k$ each, so the total memory is $O(c_{\text{in}}c_{\text{out}}k^2)$. Applying each filter to each image takes $O(k^2)$ multiplies and adds per each of hw output pixels, and we do this $c_{\text{in}}c_{\text{out}}$ times and then sum the results, so the total cost is $O(c_{\text{in}}c_{\text{out}}k^2hw)$.

2.4 2D convolution [5 points]

Suppose you have the following input

3	2	2
1	4	4
1	3	0

and you apply the following convolutional filter with zero padding, stride 1, and dilation 1:

1	4
3	2

What is the output?

22	30
26	29

2.5 Dilations [2.5 points]

Suppose the above convolution has dilation 2. What is the output in that case?

14

3 Generative modeling

3.1 MLE [2 points]

Suppose we have a set of points $x_1, \ldots, x_n \in X$ drawn i.i.d. from a distribution D whose p.d.f. we believe can be expressed by a parameterized function $f_\theta : X \mapsto \mathbb{R}_{>0}$ for some parameters $\theta \in \Theta$. Write down an optimization problem whose solution is the MLE.

$$\max_{\theta \in \Theta} \quad \sum_{i=1}^{n} \log f_\theta(x_i)$$

3.2 GAN objective [4 points]

Consider the GAN optimization problem

$$\min_{\psi} \max_{\phi} \quad \mathbb{E}_y \log g_\phi(y) + \mathbb{E}_z \log(1 - g_\phi(h_\psi(z)))$$

It has the following components:

1. generator network
2. discriminator network
3. data sample
4. Gaussian vector

Match these components to the following four terms from the optimization problem:

g_ϕ : 2
h_ψ : 1
y : 3
z : 4

3.3 What can we do with GANs? [4 points]

Describe one thing we can do after successfully optimizing the GAN objective that we cannot (directly) do after optimizing the MLE?

The GAN optimization lets us generate samples from a distribution that approximates D. To do this, sample a Gaussian vector, pass it to the generator network, and return the output of the latter.

4 Theory

4.1 VC dimension of shifted parabolas [8 points]

Consider the following hypothesis class, where each element is parameterized by a non-negative parameter $a \geq 0$.

$$H = \{h_a(x) = \text{sgn}(x^2 - a), a \geq 0\}, \qquad \text{where, } \text{sgn(z)} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0. \end{cases}$$

What is the VC dimension of H? State your answer *with a proof.*

The VC dimension of H is 1. To prove this we will observe that for any $a \leq 0$, $h_a \in H$ will classify all points in the interval $(-\sqrt{a}, \sqrt{a})$ as negative and all points outside the interval as positive.

First, we need to show that there exists at least one set of one point that can be shattered by H. Consider any $x > 0$. By choosing $a > x^2$, we can realize a negative label on this point and by choosing $a < x^2$ we can realize a positive label on this point.

Next, we will show that no set of two points can be shattered by H. Without loss of generality, let the points be $x_1 < x_2$, There are three cases we need to consider.

- First, consider $0 \leq x_1 < x_2$. In this case, we cannot realize the labeling $y_1 = 1, y_2 = 0$.
- Next, consider $x_1 < x_2 \leq 0$. In this case, we cannot realize the labeling $y_1 = 0, y_2 = 1$.
- Finally, consider the case $x_1 \leq 0 \leq x_2$. In this case, if $|x_1| \leq |x_2|$, then we cannot realize the label $y_1 = 1, y_2 = 0$ and if $|x_1| \geq |x_2|$ then we cannot realize the label $y_1 = 0, y_2 = 1$.

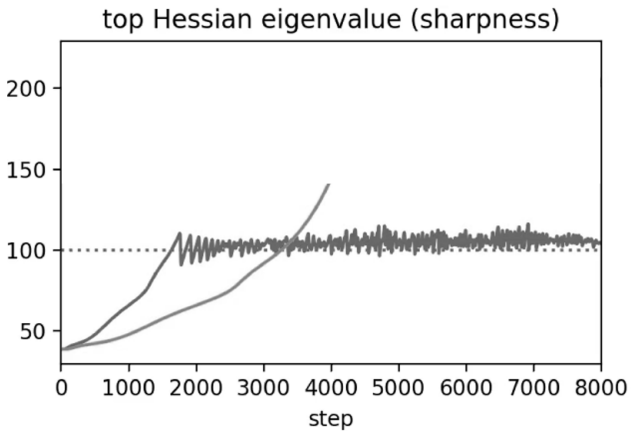
4.2 VC dimension of quadratic separators [7 points]

Now let H denote the class of quadratic separators in $\mathbb{R}^d$, i.e. functions of form

$$h(w) = \text{sgn}\left(b + \sum_{i=1}^{d} w_i x_i + \sum_{i=1}^{d} \sum_{j=i}^{d} w_{ij} x_i x_j\right)$$

for parameters b, w_i, w_{ij}. Show that the VC dimension of H is $O(d^2)$. You may use without proof the fact that the VC dimension of linear separators in $\mathbb{R}^n$ is $n + 1$. Hint: you only need to show an upper bound.

Define $n = d + (d + 1)d/2$ and to each point $x \in X$ associate a point $\phi(x) \in \mathbb{R}^n$ s.t. the first d entries correspond to $x_1, \ldots, x_d$ and the last $(d + 1)d/2$ entries correspond to x_{ij} for $i = 1, \ldots, n$ and $j = i, \ldots, n$. Now suppose the VC dimension of H is $> n + 1$. Then H shatters a subset $X \subset \mathbb{R}^d$ of size $n + 2$, i.e. there are settings of b, w_i, w_{ij} that can correctly classify any labeling of X. Then the set of points $\{\phi(x) : x \in \mathcal{X}\} \subset \mathbb{R}^n$ of size $n + 2$ is shattered by the set of n-dimensional linear separators, since any labeling of the points $\phi(x), x \in X$ will be correctly classified by the linear separator with the intercept b and coefficients w_i, w_{ij} corresponding to the parameters of the quadratic separator that correctly classifies the points $x \in X$. This is a contradiction since the VC dimension of linear separators is $n + 1$ and so there cannot exist a set of points of size $n + 2$ that is shattered by n-dimensional linear separators. Thus the VC-dimension of quadratic separators cannot be greater than $n = d + (d + 1)d/2 = O(d^2)$, as desired.



5 Optimization

According to classical optimization theory, I should pick the largest possible step-size for which gradient descent is stable, and gradient descent is unstable if the step-size $\alpha > 2/L$, where L is the sharpness. Above is a plot of the local sharpness of neural network being optimized using gradient descent with step-size $\alpha = 0.02$ across 8000 gradient steps, as well as the first 4000 gradient steps of the same curve but for gradient descent with step-size $\alpha = 0.01$.

5.1 Edge of stability [4 points]

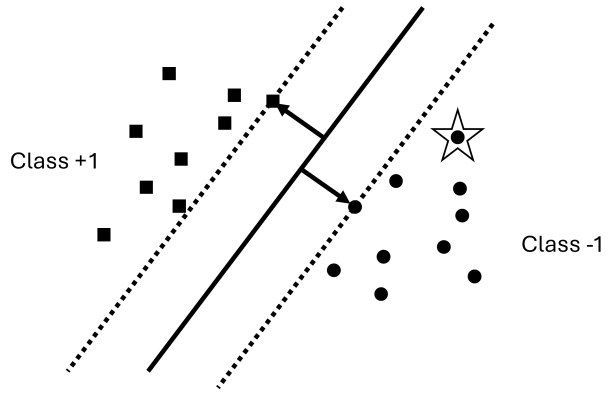
Make a rough sketch of the trajectory of the local sharpness for the remaining steps.

The plot should continue the curve until it reaches 200 on the y-axis and then start oscillating around that threshold until 8000.

5.2 Optimization guarantees [4 points]

What predictably happens when I pick a fixed step-size $\alpha > 0$ and optimize a neural network using gradient descent, and what does this say about the utility of classical optimization theory for picking the step-size in neural networks?

No matter what step-size α I use, the gradient descent trajectory will reach the sharpness threshold $L = 2/\alpha$ and oscillate around it while continuing to minimize the training objective. This means the classical theory is not useful for picking the step-size.



6 Marginalia

6.1 SVM [3 points]

Suppose I train a hard-margin linear SVM on the above data and obtain the linear classifier denoted by the solid line, with margin denoted by the arrows and dotted line. What happens when I remove the starred point from the dataset and retrain the model on the remaining data, and why?

The point is not a support vector (on the margin) and so the resulting linear classifier will be the same.

6.2 Separability [4 points]

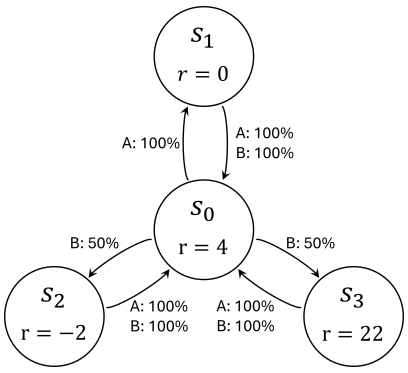
Suppose my data is not linearly separable. What are two things I can do to run SVM in that case?

- Add slack variables and run soft-margin SVM.
- Use kernel SVM with a high-order kernel (e.g. polynomial with degree ≥ 2) and hope the dataset is separable in the higher dimensional space.

6.3 Active learning [3 points]

Explain why the notion of margin is useful in active learning.

In active learning we must select points to label, and it is often more useful to pick harder-to-classify points. Margin relative to an existing classifier is one way of measuring the difficulty of a point.



7 Reinforcement learning

Consider the above MDP where starting state s_0 has two actions: A takes us to state s_1 with 100% probability, while B takes us to either s_2 or s_3 with equal probability. In all other states all actions take us back to s_0 with 100% probability. The reward is 4 in s_0, 0 in s_1, -2 in s_2, and 22 in s_3. Set a discount factor of $\gamma = 0.5$.

7.1 Optimal policy [4 points]

What is an optimal policy for this MDP? Argue briefly for your answer.
An optimal policy is $\pi^*(s_i) = B \; \forall \; i$, as in state s_0 it will give us expected reward $4+\gamma(-2\times0.5+22\times0.5) = 9$ and return to the start, while A only gives expected reward 3. The actions in the other states do not matter.

7.2 Value function [6 points]

Compute the optimal value function for each state.
π^* gets the reward 4 from s_0 every other state starting at $t = 0$ and an expected reward 10 every other state starting at $t = 1$. To handle this we square the discount factor for both rewards, discount the second by one step, and add them up to get $V^*(s_0)$. The remaining values can be derived from $V^*(s_0)$.

- $V^*(s_0) = \sum_{t=0}^{\infty} \gamma^{2t} 4 + \gamma \sum_{t=0}^{\infty} \gamma^{2t} 10 = \frac{4+10\gamma}{1-\gamma^2} = 12$
- $V^*(s_1) = 0 + \gamma V^*(s_0) = 6$
- $V^*(s_2) = -2 + \gamma V^*(s_0) = 4$
- $V^*(s_3) = 22 + \gamma V^*(s_0) = 28$

7.3 Q-learning [5 points]

Suppose I initialize my Q-table to $Q(s_i, a) = 0$ for all $i \in \{0, 1, 2, 3\}$ and $a \in \{\text{A,B}\}$, and then I run Q-learning with step-size $\alpha > 0$ while using the greedy policy, i.e. $\pi(s) = \arg\max_a Q(s, a)$, splitting ties uniformly at random. Show that with probability at least 50% I never visit state s_3.
With 50% probability the first action is A, resulting in $Q(s_0,\text{A}) = 4\alpha$ and moving to s_1. w.l.o.g. the next action is A, resulting in $Q(s_1,\text{A}) = \alpha\gamma \max_a Q(s_0, a) = 4\alpha^2\gamma$. At all future times, any time action A is taken the Q-learning updates in states s_0 and s_1 will be $Q(s_0,\text{A}) \leftarrow (1-\alpha)Q(s_0,\text{A}) + 4\alpha + \gamma \max_a Q(s_1, a)$ and $Q(s_1,\text{A}) \leftarrow (1-\alpha)Q(s_1,\text{A}) + 4\alpha + \gamma \max_a Q(s_1, a)$, respectively, so both $Q(s_0,\text{A})$ and $Q(s_1,\text{A})$ will stay positive. Since all other entries are zero, action A will be taken always, so we will never visit state s_3.

7.4 Mitigations [2 points]

What can we do to avoid the above situation where we end up with a suboptimal policy?
Force exploration using ε-greedy that sometimes takes suboptimal (according to our current Q-table) actions.

8 Transfer learning

8.1 Different tasks [2 points]

What is the difference between pretext tasks and multi-task learning tasks?

Pretext tasks are typically made up tasks that use information about the inputs to induce trivial-to-obtain labels; we only care about them insofar as they help us do well on the downstream task. On the other hand, the objective of multi-task learning is to do well on all tasks.

8.2 Meta-learning [4 points]

Describe one similarity AND one difference between pretext tasks and meta-training tasks.

Both tasks are used to learn something that helps us do well on a future task, but meta-training tasks typically have supervised labels in the same label space as the downstream task whereas pretext tasks typically have self-supervised labels that are entirely unrelated to the downstream task.

8.3 Foundation models [4 points]

Suppose you want to do sentiment analysis (given a text string, predict whether it is positive or negative) with a small amount of data. You download a foundation model trained to do next-word prediction, but your GPU only has enough memory to do forward passes with the model, not backward passes (backpropagation). State and describe two techniques that you could use to still do supervised learning on your data.

Any two of the following:

- feature extraction: replace the next-word prediction layer of the FM with a light binary classifier (e.g. a linear model or MLP), freeze all the other model parameters, and train on your data.
- parameter-efficient fine-tuning or LoRA: use a technique that reduces the number of parameters that require backpropagation, e.g. by replacing the weight matrices by a low-rank product.
- in-context learning: for each test string, pass a set of instructions, the labeled pairs, and the test string as a prompt to the model and then generate the predicted label using next-word prediction.